

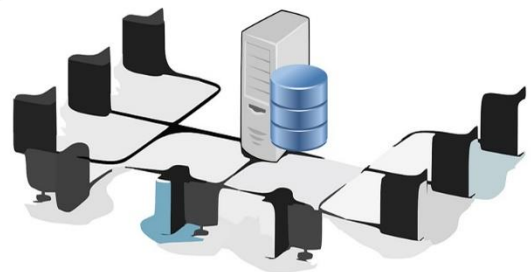
Question:

Explain the difference between Centralized database, Client/Server Database and Distributed database and give an example of each.

Centralized Database :

A centralized database is stored and maintained in a single location, usually a central computer or server. All users access this one central database through terminals or network connections. The advantage is that data management, security, and backup are easier because everything is in one place. However, if the central server fails, the entire system goes down, and heavy traffic can slow performance.

CENTRALIZED DATABASE



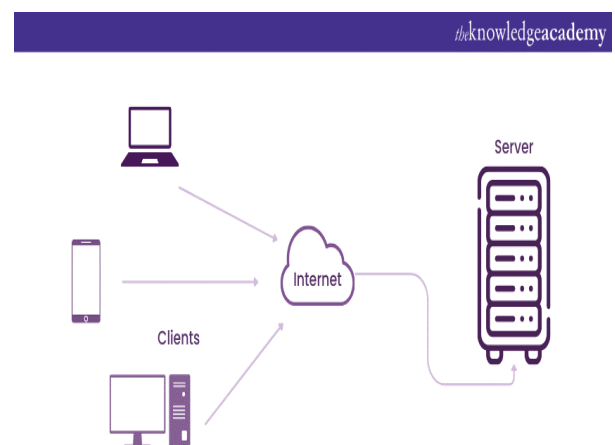
Example :

A university storing all student records, courses, and teacher information in a single central server on campus.

Client/Server Database :

In a client/server database, the system is divided into two main parts: the server, which manages and stores the database, and the clients, which are applications or user interfaces that request services from the server. This allows multiple users to access the database from different computers while the server handles queries and updates.

There are three types of client server architecture described below:



Single-Tier Database Architecture :

In a single-tier architecture, the database and the user interface both exist on the same machine. There is no separation between the client and the server — the user directly interacts with the database. Basically, the user can sit at one computer, open the

database system, and directly access, update, or manage the data without needing any network connection or middle layer.

The main advantage is simplicity. Since everything runs on the same system, it is faster for small-scale use, and there are no network delays. But the disadvantage is that it is not suitable for multiple users or large organizations. It lacks security, scalability, and centralized control, which is why it is only used for personal or small applications.

Example :

MS Access or a personal database created in Microsoft Excel on your own computer is a good example of a single-tier database.

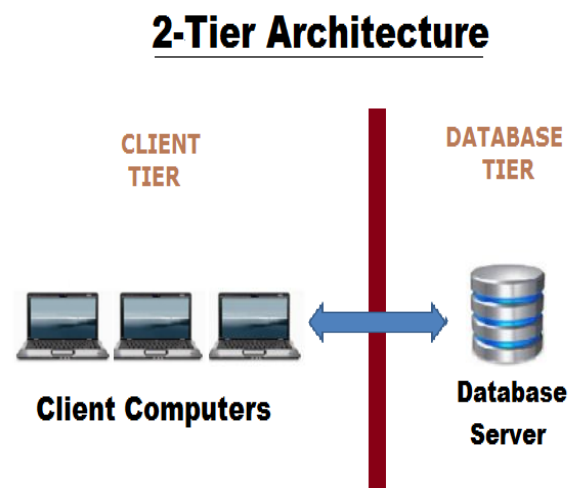


Two-Tier Database Architecture :

In a two-tier architecture, the client directly communicates with the database server. The client sends queries (like SQL commands) to the server, and the server processes them and returns the results. The client is responsible for presenting the data to the user, while the server manages storage and processing. This model is simple and faster for small applications, but it puts heavy load on the server when many clients connect at the same time.

Example :

A desktop library management system where all computers in a lab directly connect to a single database server.

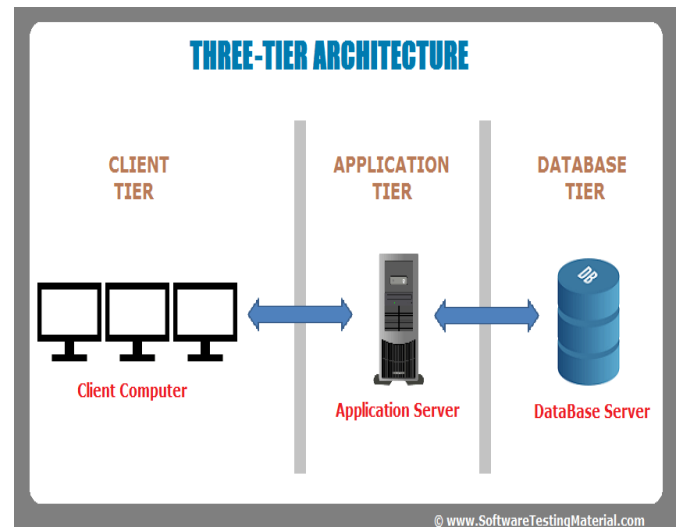


Three-Tier Database Architecture :

In a three-tier architecture, there is an additional middle layer called the application server between the client and the database server. The client only communicates with the application server, which handles business logic, processing, and security before interacting with the database. This makes the system more secure, scalable, and easier to manage, since clients don't directly access the database. However, it is more complex to design compared to the two-tier model.

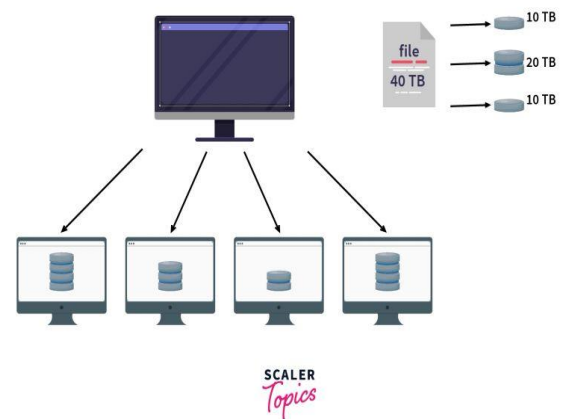
Example :

Online banking systems where the user's app communicates with an application server, and that application server talks to the bank's database.



Distributed Database :

A distributed database is not kept in a single place but is spread across multiple computers or sites connected through a network. Each site may hold a portion of the database, and in some cases, copies of the same data are replicated in different locations. The benefit is reliability, faster access for users near different sites, and fault tolerance. However, managing and synchronizing data across all sites can be complex.



Example :

Google's search engine database, which is distributed across data centers worldwide to provide fast results to users from any location.

1. Homogeneous Distributed Database

In a homogeneous distributed database, all the sites use the same DBMS software, data structures, and operating system. Since everything is uniform, the system looks like a single database to the user even though it is spread across different locations. This makes communication and management easier but reduces flexibility.

Example: A company using Oracle DBMS at multiple branches, all connected together in a distributed setup.

2. Heterogeneous Distributed Database

In a heterogeneous distributed database, the sites may use different DBMS software, data models, or operating systems. Special translation and integration software is

needed so that these different systems can communicate and work together. It is more flexible but harder to manage because of incompatibility issues.

Example: A multinational company where one branch uses Oracle, another uses MySQL, and another uses SQL Server, but all are connected in a distributed way.